Artframe

A full-stack e-commerce web application dedicated to selling high-quality aesthetic, cultural, and decorative frames.

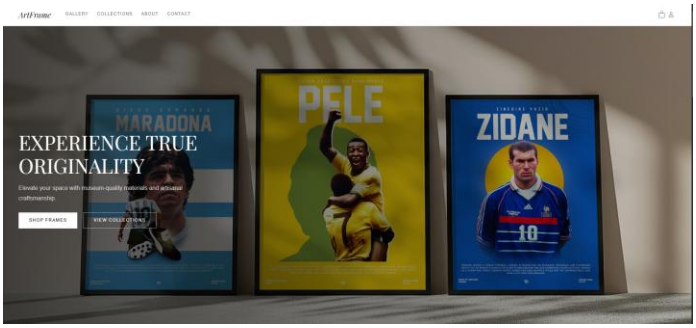
Motivation

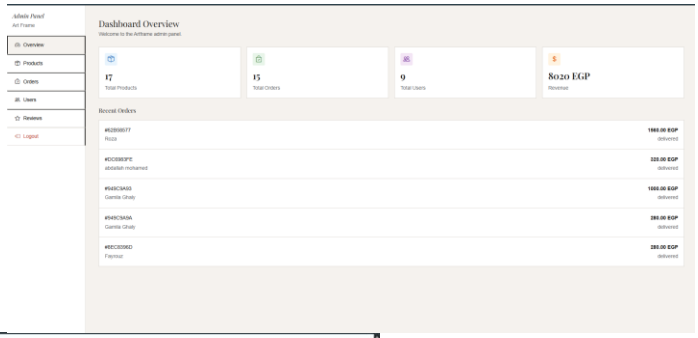
This project was built to provide a seamless, highly secure shopping experience for art enthusiasts. The goal was to engineer a robust, scalable backend capable of dynamic cart calculations, real-time inventory management, and role-based access control, seamlessly paired with a modern, responsive React frontend.

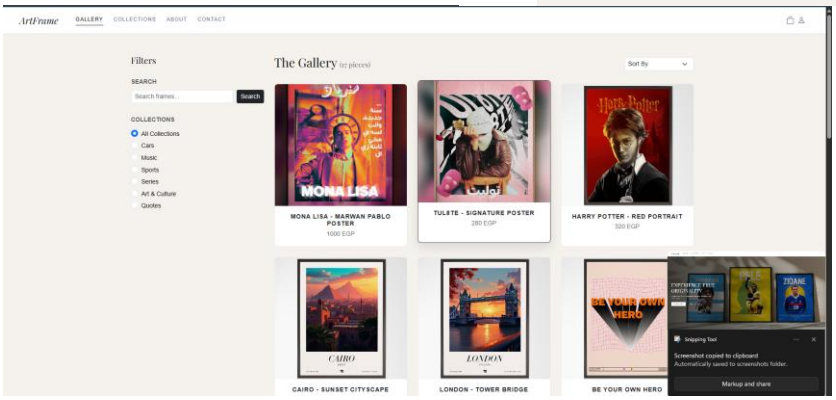
Build Status

Active / MVP Complete. The RESTful API and core frontend integrations are fully operational. Current focus is on UI polish and final deployment preparations.

Screenshots







Tech / Framework Used

Frontend Ecosystem:

- **React 18:** Single Page Application (SPA) architecture.
- **React Router v7:** Client-side routing with custom route guards.
- **Context API:** Global state management (Auth & Cart) without Redux bloat.
- **Bootstrap 5:** Responsive grid layout and UI components.
- **Formik:** Advanced form handling and validation.

Backend Ecosystem:

- **Node.js & Express.js:** Core server and REST API framework.
- **MongoDB & Mongoose:** NoSQL database and Object Data Modeling (ODM).
- **JWT & bcrypt:** Stateless authentication, password hashing, and server-side pepper security.
- **Morgan:** Request logging and API debugging.
- **CORS & Dotenv:** Cross-origin resource sharing and environment variable management.

Key Features

- **Role-Based Access Control (RBAC):** Secure user and admin routes enforced by both backend JWT middleware and frontend React wrapper components.
- **Smart Cart System:** Real-time, database-driven cart totals calculated on-the-fly (`.reduce()`) to ensure absolute price accuracy. Cart state automatically hydrates via `useEffect` upon user login.
- **Inventory Management:** Automated stock depletion when an admin marks an order as 'delivered', utilizing `Math.max` fail-safes to prevent negative inventory.
- **Dynamic Search & Filtering:** Case-insensitive product searching (`$regex`) and category-based filtering built directly into the API query parameters.
- **Relational NoSQL:** Extensive use of Mongoose `.populate()` to minimize frontend API requests and keep the database normalized.
- **Automated Database Seeding:** The backend automatically bootstraps default frame categories on startup to improve the developer experience.

Code Example

Here is a look at how the frontend dynamically guards secure routes by consuming the global Authentication Context:

```
// src/components/ProtectedRoute.js

import { Navigate, Outlet } from "react-router-dom";

import { useAuth } from "../context/AuthContext";



export function AdminRoute() {

  const { user, token } = useAuth();



  // Kick unauthenticated users to login

  if (!token) return <Navigate to="/login" replace />;



  // Kick non-admins back to the storefront

  if (user?.role !== "admin") return <Navigate to="/" replace />;



  // Render the protected dashboard

  return <Outlet />;

}
```

Installation & Setup

To run Artframe locally, you will need Node.js and a MongoDB instance (local or Atlas) installed.

1. Clone the repository

```
git clone [https://github.com/ziadshady15/E-Commerce-App/tree/main/e-commerce-app)
```

2. Backend Setup

Open a terminal and navigate to the backend folder:

```
cd backend

npm install
```

Create a `.env` file in the root of the `backend` folder with the following variables:

```
PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_super_secret_jwt_key
```

Start the backend server:

```
npm run dev
```

3. Frontend Setup

Open a new terminal window and navigate to the frontend folder:

```
cd frontend

npm install
```

Create a `.env` file in the root of the `frontend` folder with the following variable:

```
REACT_APP_API_URL=http://localhost:5000/api
```

Start the React development server:

```
npm start
```

License

MIT © [Artframe]